ABSTRACTION

Definition:

The process of hiding the method implementation (method block) and displaying only the feature (method declaration) is called as Abstraction.

Abstraction is a designing technique, used in the presentation layer.

Data Hiding using OOPS:

- Attributes are hidden using Encapsulation.
- Method implementation is hidden using Abstraction.

TWO COMPONENTS:

To achieve abstraction in Java, two components of Java are used.

1. Abstract Class
2. Interface

Both these components ***cannot be instantiated (no object creation)***.

TWO TECHNIQUES:

1. Inheritance or Is-A Relationship
2. Polymorphism – Run Time – Method Over Riding

ABSTRACT CLASS:

A class in Java can be made as abstract class by,

- Using the abstract modifier/keyword before 'class' keyword
- Using at least on abstract method in the class

Abstract method:

The syntax for abstract method looks like this:

```
[access modifier] abstract return_type method_name ( [formal arguments] );
```

All the methods we covered so far are methods with body/block. Those methods are called **concrete methods**.

Methods **without body/block/implementation** are called abstract methods.

Characteristics of Abstract method:

- Declaration/Feature is present
- Block/Implementation is absent
- Implementation is given by concrete method with same signature present in its own child class

Abstraction using Abstract class:

```java

class AbstractionClass
{
    public static void main(String[] args)
    {
        //generalization
        Hero h = new Passion();
        //down casting not needed in over riding
        h.service();
        //specialization
        Passion p = new Passion();
        p.service();
        h.mileage();
// abstract class allows concrete non static methods
// abstraction is not 100%
    }
}

abstract class Hero
{
    String color;
    int price;
    public void mileage() // non static concrete method
    {
        System.out.println("mileage got tuned");
    }
    abstract public void service(); // non static abstract method
}

class Passion extends Hero
{
    public void service()
    {
        System.out.println("you have three free services");
    }
}

```

```
<terminated> AbstractionClass [Java Application] C
you have three free services
you have three free services
mileage got tuned
```

INTERFACE:

An Interface is a component of Java similar to a class.

The syntax of an interface looks like this:

```
interface Interface_Name
{
}
```

Members of an Interface:

An interface can have only three type members as follows:

1. Single Line Static Initializers which are final by default
2. Static methods which are concrete
3. Non-Static methods which are abstract

Advantages of Interface:

1. ***To achieve 100% abstraction***
   Abstract class allows non-static concrete methods, which means implementation of all non-static methods are not hidden.
2. ***To achieve multiple inheritance in Java with same methods in parents***
   If both parents are interfaces, then multiple inheritance can be performed since only child class has implementation to inherited non-static method.

Inheritance using Interface:

- ❖ When both parent and child are interfaces, extends keyword is used.
- ❖ When parent is interface and child is class, implements keyword is used.
- ❖ Interface cannot be a child to a class in Java.

Abstraction using Interface:

```
 2
 3 class AbsInterfaceDiamond implements InterfaceA, InterfaceB
 4 {
 5     public static void main(String[] args)
 6     {
 7         InterfaceA ia = new AbsInterfaceDiamond();
 8         ia.xyz();
 9
10         InterfaceB ib = new AbsInterfaceDiamond();
11         ib.xyz();
12
13         InterfaceA.abc();
14         // ACCESSING STATIC CONCRETE METHOD OF INTERFACE
15     }
16     public void xyz()
17     {
18         System.out.println("xyz method from driver class");
19     }
20 }
21
22 interface InterfaceA
23 {
24     final static int a = 10; // it is final by default
25     public static void abc() // static members are not inherited
26     {
27         System.out.println("static concrete method");
28     }
29     abstract public void xyz();
30     // it is abstract by default, abstract non static method
31 }
32
33 interface InterfaceB
34 {
35     public void xyz(); // abstract non static method
36 }
37
```

```
<terminated> AbsInterfaceDiamond [Java Applicatio
xyz method from driver class
xyz method from driver class
static concrete method
```

Functional Interface:

An interface which has only one member – only one abstract non-static method is called as functional interface. In the above example, InterfaceB is a functional interface.